

CYBERSECURITY TOOLS · WEEKLY LEARNING TRACK

Nmap Reference Documentation

Target Specification & Host Discovery (Complete)

Prepared by Emmanuel Aidoo

Kali Linux · Nmap 7.98
September 14–16, 2026

Contents

- 1 Single IP Address Scan..... 3
- 2 Multiple Host Scanning 4
- 3 CIDR Range Scan 5
- 3a Host Discovery Techniques (-sn and -Pn)..... 6
- 3b Enumerating Targets Only (-sL / List Scan) 7
- 4 Verbose Scan & Timing Behaviour 9
- 5 Key Takeaways 10

Objective: To get comfortable with how Nmap accepts targets, from a single IP up through a full CIDR range, and see what its scan phases and timing engine actually look like in practice.

Topic: single-IP scanning, multi-host scanning, CIDR ranges, host discovery (-sn, -Pn, -sL), and Nmap's internal scan phases and timing behaviour.

1 Single IP Address Scan

```
$ nmap 173.230.138.70
```

What I learned

A scan against one target IP with default Nmap settings (SYN scan, top 1000 ports).



Figure 1 • Default scan of a single host (173.230.138.70)

Result summary

Port	State	Service
22/tcp	open	ssh
80/tcp	open	http
587/tcp	open	submission
3306/tcp	closed	mysql

My observation

996 of the 1000 scanned ports were filtered (no response). The host resolved to 173-230-138-70.ip.linodeusercontent.com, confirming it's a Linode-hosted server. Total scan time: 11.64 seconds.

2 Multiple Host Scanning

```
$ nmap 8.8.8.8 173.230.138.70
```

What I learned

Nmap accepts more than one target in a single invocation. I just have to separate them with spaces. Each target is scanned independently and reported in its own block.

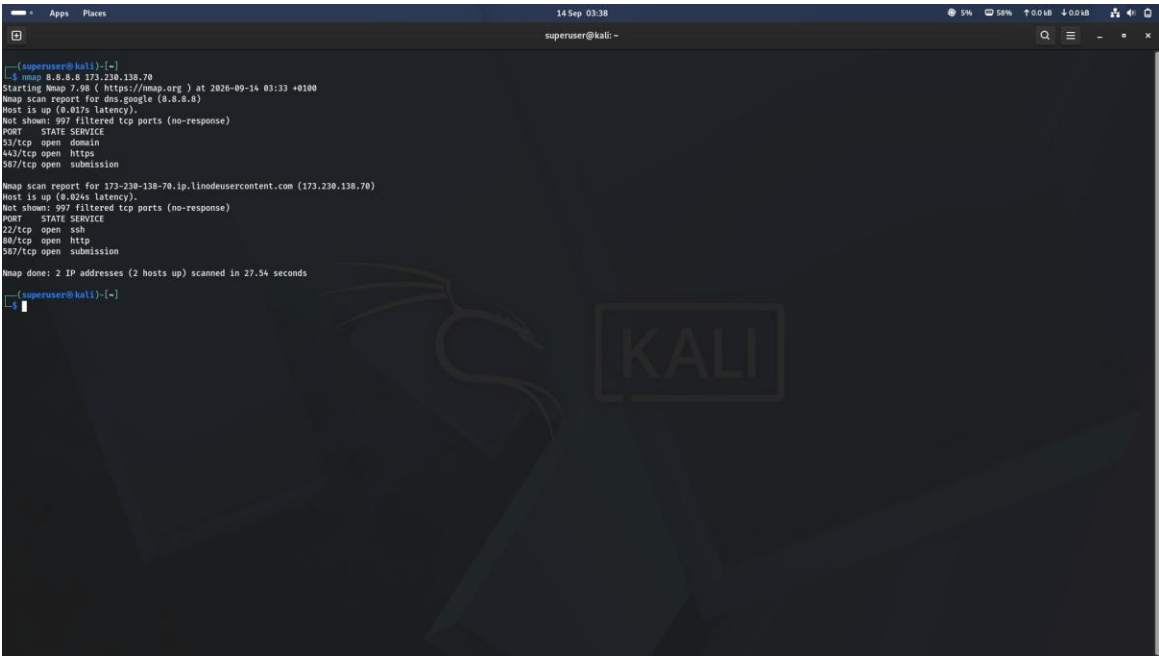


Figure 2 · Scanning two unrelated hosts in one command

Result summary

Host	Open Ports	Notes
8.8.8.8 (dns.google)	53, 443, 587	Google public DNS resolver
173.230.138.70	22, 80, 587	Same Linode host as Section 1

My observation

Nmap ran the two scans back-to-back and reported “2 IP addresses (2 hosts up) scanned in 27.54 seconds”. Roughly double the time of the single-host scan, since each target still gets its own full port probe.

3 CIDR Range Scan

```
$ nmap 10.0.2.15/24
```

What I learned

Appending a CIDR suffix (e.g. /24) tells Nmap to expand the target into every address in that subnet and scan each one that responds. This is the fastest way to sweep a whole local network.

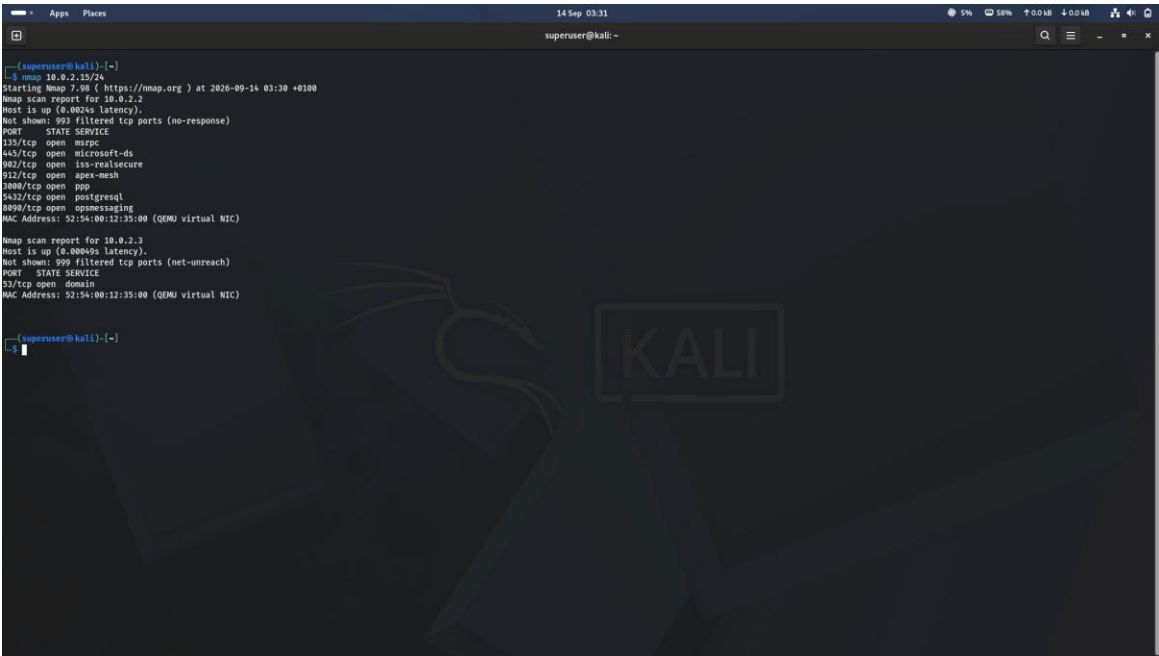


Figure 3 · CIDR sweep of the 10.0.2.0/24 lab network

Result summary

Host	Open Ports	Notes
10.0.2.2	135, 445, 902, 912, 3000, 5432, 8090	Likely the VMware/VirtualBox NAT gateway — a wide service mix (RPC, SMB, VMware tools, Postgres) is typical of the host machine's NAT interface, not a real endpoint
10.0.2.3	53 (domain)	The NAT DNS relay provided by the hypervisor

My observation

Both hosts share the same MAC address (52:54:00:12:35:00), confirming they're virtual interfaces on the same QEMU/KVM NAT segment rather than separate physical machines.

3a Host Discovery Techniques (-sn and -Pn)

```
$ nmap -sn 10.0.2.15/24
```

What I learned

The `-sn` flag tells Nmap to skip port scanning entirely and only determine which hosts on a target range are alive. It's the fastest way to map a subnet before deciding which specific hosts are worth a full port scan.

Result summary

Host	Status	Notes
10.0.2.2	Up (0.0018s)	NAT gateway — same host seen in the Section 3 CIDR port scan
10.0.2.3	Up (0.00058s)	NAT DNS relay — same host seen in the Section 3 CIDR port scan
10.0.2.15	Up (no latency shown)	The scanning machine itself (Kali VM) — Nmap doesn't measure round-trip time to its own interface

My observation

The Section 3 scan (“`nmap 10.0.2.15/24`”, no flag) ran full 1000-port probes and only surfaced .2 and .3 as reportable hosts. This `-sn` sweep found a third host — 10.0.2.15, the Kali box running the scan — and finished all 256 addresses in 7.11 seconds versus far longer for a full port scan. That speed comes from skipping the port-probing phase: on a local subnet, `-sn` relies on quick ARP requests instead of sending TCP SYN packets to up to 1000 ports per host.

Skipping host discovery entirely (-Pn)

What I learned

`-Pn` takes the opposite approach: instead of skipping the port scan, it skips the host-discovery (ping) phase and simply assumes every target address is online, going straight to port scanning each one. This matters because many real-world hosts are configured to drop ICMP/ping probes — without `-Pn`, Nmap would wrongly report them as down and skip scanning them entirely.

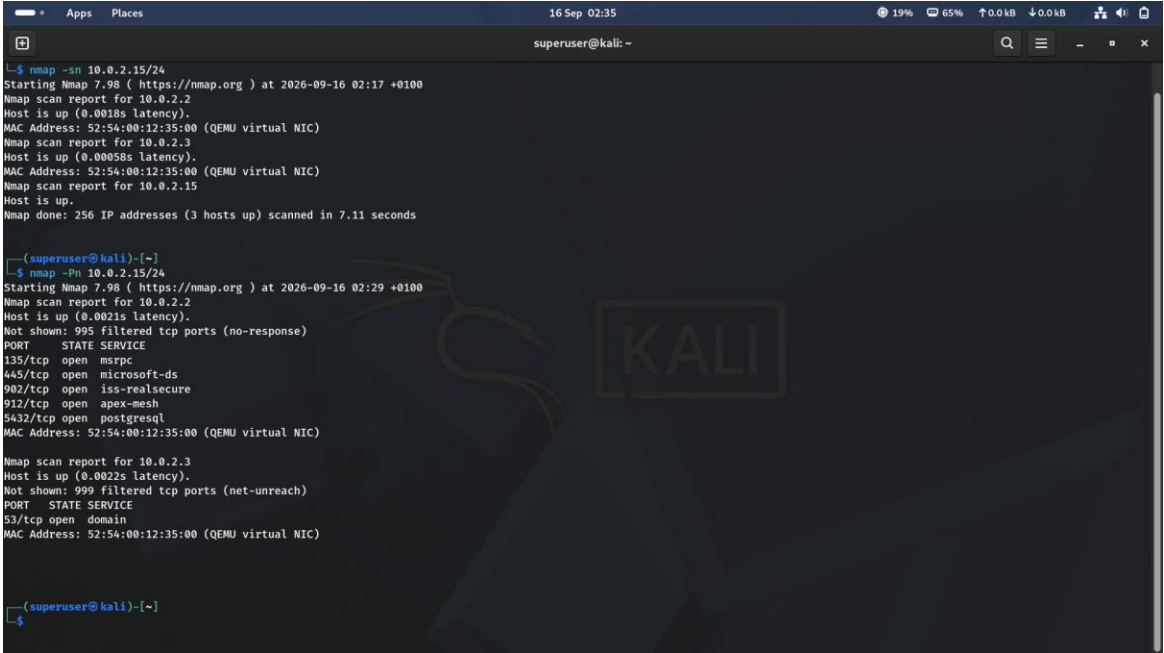


Figure 4 • Ping sweep (-sn) followed by a no-ping scan (-Pn) against the same lab network

Result summary

Host	Open Ports	Notes
10.0.2.2	135, 445, 902, 912, 5432	Same NAT gateway — but only 5 ports open this time (no 3000 or 8090), a reminder that ephemeral local services can start and stop between scans
10.0.2.3	53 (domain)	NAT DNS relay, consistent with every earlier scan of this host

My observation

-sn answers “who's alive?” without touching ports. -Pn answers “what's open?” without trusting the ping step — use it against firewalled or ICMP-blocking targets where a normal scan might falsely report the host as down.

3b Enumerating Targets Only (-sL / List Scan)

```
$ nmap -sL 10.0.2.0/24
```

What I learned

-sL goes a step further than -sn: it doesn't send a single packet to any target. It simply expands the target specification into a full list of addresses and attempts a reverse-DNS lookup on each one — nothing more. No

ping, no ARP, no ports. It's the safest possible way to sanity-check what a CIDR range or target list actually expands to before running anything that touches the network.



Figure 5 · List scan enumerating every address in 10.0.2.0/24 without probing any of them

Result summary

Aspect	Observation
Addresses listed	All 256 addresses in the /24, from 10.0.2.0 through 10.0.2.255, printed sequentially
Reverse DNS	None resolved for the visible range — expected, since this is a private NAT subnet with no PTR records configured
Host status	Not reported at all — unlike -sn or a full scan, -sL never determines whether a host is up

My observation

Think of these three as a spectrum of how much a scan touches the network. -sL touches nothing — pure enumeration, zero packets sent. -sn touches the network lightly, just enough to ask “is anyone home?” -Pn touches it the most among these three, skipping the ask-first step and probing ports on every address regardless. Use -sL first when I just want to preview or double-check a target range — for example, confirming a /24 really does expand to the 256 addresses I expect — before committing to any live scan.

4 Verbose Scan & Timing Behaviour

```
$ nmap scanme.nmap.org
```

What I learned

Running a scan against Nmap's public test host, scanme.nmap.org, surfaces the scan phases and timing engine that are normally hidden in quiet output. Three phases run in sequence: DNS resolution, a quick ping scan to confirm the host is up, and finally the full SYN stealth scan across 1000 ports.

```

superuser@kali: ~
--(superuser@kali)~$-
$ nmap scanme.nmap.org
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-14 03:16 +0100
Verbosity increased to 1.
Completed Parallel DNS resolution of 1 host. at 03:16, 2.38s elapsed
Initiating Ping Scan at 03:16
Scanning scanme.nmap.org (45.33.32.156) [4 ports]
Completed Ping Scan at 03:16, 0.01s elapsed (3 total hosts)
Initiating Parallel DNS resolution of 1 host. at 03:16
Completed Parallel DNS resolution of 1 host. at 03:16, 2.36s elapsed
Initiating SYN Stealth Scan at 03:16
Scanning scanme.nmap.org (45.33.32.156) [1000 ports]
Discovered open port 587/tcp on 45.33.32.156
Discovered open port 22/tcp on 45.33.32.156
Discovered open port 80/tcp on 45.33.32.156
Discovered open port 22/tcp on 45.33.32.156
Discovered open port 80/tcp on 45.33.32.156
Increasing send delay for 45.33.32.156 from 0 to 5 due to 11 out of 17 dropped probes since last increase.
adjust_timeouts2: packet supposedly had rtt of 9047889 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 9047889 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 9047865 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 9049656 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 9049656 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14034100 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14034100 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14033771 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14033771 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14034723 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14034723 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14039200 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 14039200 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022549 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022549 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8019543 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8019543 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022681 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022681 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8018715 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8028681 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8028681 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022554 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8022554 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8020950 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8020950 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8020957 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8020957 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8026212 microseconds. Ignoring time.
adjust_timeouts2: packet supposedly had rtt of 8026212 microseconds. Ignoring time.
Discovered open port 9929/tcp on 45.33.32.156
Increasing send delay for 45.33.32.156 from 5 to 10 due to max_successful_tryno increase
--(superuser@kali)~$-

```

Figure 6 · Scan phases and adaptive timing against scanme.nmap.org

Result summary

- DNS resolution — scanme.nmap.org resolved to 45.33.32.156 before any probing started.
- Ping Scan — a quick host-discovery pass confirmed the target was up (1 host, 4 ports probed) before committing to a full port scan.
- SYN Stealth Scan — the main 1000-port probe. Ports 22, 80, 587 and later 9929 were discovered open.

My observation

The repeated 'adjust_timeouts2' lines show Nmap's round-trip-time estimator reacting to inconsistent or dropped responses over the internet link (as opposed to the near-zero latency of the local lab network in Section 3). The first jump — send delay increased from 0 to 5 — was explicitly triggered by packet loss: 11 of the last 17 probes were dropped. The second jump, from 5 to 10, was logged as triggered by a rising successful-try count rather than fresh drops. Either way, this is built-in congestion-control behaviour, not an error. Scans against real internet hosts are noisier and slower than scans on a local subnet, and Nmap's timing engine, not a fixed delay I set, adapts automatically to keep the scan reliable.

5 Key Takeaways

- A bare IP, hostname, list of targets, or CIDR block are all valid ways to specify what Nmap scans — the syntax scales from one host to a whole subnet.
- Filtered vs. closed vs. open all mean different things: filtered = no response (likely firewalled), closed = host responded but nothing is listening, open = a service is listening.
- Shared MAC addresses across 'different' hosts is a strong signal you're looking at virtual NAT infrastructure, not distinct machines.
- Nmap's timing/retransmission engine self-adjusts based on real network conditions — this is normal and expected on higher-latency or lossy links.
- -sn (ping sweep) skips port probing entirely, so it's the fastest way to find live hosts on a subnet before running a full, targeted port scan on just those hosts.
- -Pn skips the opposite step — host discovery — and scans ports on every target regardless of ping response, which is essential against hosts that silently drop ICMP probes.
- -sL sends zero packets to any target — it only enumerates the address list and attempts reverse DNS, making it the safest way to preview a range before scanning it for real.